Highlights

Gated Rotary-Enhanced Linear Attention for Long-term Sequential Recommendation

Juntao Hu, Wei Zhou, Huayi Shen, Xiao Du, Jie Liao, Min Gao, Jun Zeng, Junhao Wen

- Rotary Enhanced Linear Attention integrates RoPE for efficient long-sequence modeling.
- Local Shortcut operation injects dense local user preferences via causal convolution.
- RecGRELA achieves competitive performance with low memory overhead.

arXiv:2506.13315v2 [cs.IR] 2 Nov 2025

Gated Rotary-Enhanced Linear Attention for Long-term Sequential Recommendation

Juntao Hu[a], Wei Zhou[a], Huayi Shen[a], Xiao Du[a], Jie Liao[a], Min Gao[a], Jun Zeng[a] and Junhao Wen[a,*]

ARTICLE INFO

Keywords:
Sequential Recommendation
Linear Attention
Rotary Position Encoding
Gated Mechanism

ABSTRACT

In Sequential Recommendation Systems (SRSs), Transformer models have demonstrated remarkable performance but face computational and memory cost challenges, especially when modeling long-term user behavior sequences. Due to its quadratic complexity, the dot-product attention mechanism in Transformers becomes expensive for processing long sequences. By approximating the dot-product attention using elaborate mapping functions, linear attention provides a more efficient option with linear complexity. However, existing linear attention methods face three limitations: 1) they often use learnable position encodings, which incur extra computational costs in long-term sequence scenarios, 2) they may not sufficiently account for user's fine-grained local preferences (short-lived burst of interest), and 3) they try to capture some temporary activities, but often confuse these with stable and long-term interests. This can result in unclear or less effective recommendations. To remedy these drawbacks, we propose a long-term sequential **Rec**ommendation model with **G**ated **R**otary **E**nhanced **L**inear **A**ttention (**RecGRELA**). Specifically, we first propose a Rotary-Enhanced Linear Attention (RELA) module to efficiently model long-range dependency within the user's historical information using rotary position encodings. Then, we introduce a local short operation to add the local preferences of interactions and show the theoretical insight. We further introduce a SiLU-based Gated mechanism for RELA (GRELA) to help the model tell if a user behavior shows a short-term, local interest or a real change in their long-term tastes. Experimental results on four public benchmark datasets show that our RecGRELA achieves state-of-the-art performance compared with existing SRSs based on Recurrent Neural Networks, Transformer, and Mamba while keeping low memory overhead. Our source code is available at https://anonymous.4open.science/r/RecGLARE-B4AC.

1. Introduction

Sequential Recommender Systems (SRSs) [54, 9, 37, 35, 68, 48] have become a fundamental technology in personalized services, significantly transforming how online platforms predict user behavior. By analyzing sequences of user interactions across e-commerce, streaming services, and social media, SRSs capture both short-term interests and long-term preference trends. However, modeling long-term dependencies is challenging, as efficiently handling long sequences demands a balance between computational cost and model performance, while also capturing the dynamic and complex patterns in user interests.

To address the challenge of modeling user behavior sequences, researchers explore various methodologies. Traditional approaches, such as those based on Markov Chains [17, 42] or Recurrent Neural Networks (RNNs) [1, 3, 4, 58], lay the groundwork but also face limitations in capturing long-range dependencies or suffered from computational inefficiencies during training and inference, particularly with increasing sequence lengths. Consequently, the field moves towards more advanced models that can handle complex sequential dependencies more effectively. This also presents new challenges.

Specifically, the first challenge is ***Efficiency***. Transformer models [19, 47, 63] emerge as a compelling replacement for earlier approaches like RNNs, leveraging their ability to process sequential data and capture both local and global preferences of users. However, the dot-product attention mechanism in Transformers exhibits quadratic complexity relative to sequence length, resulting in high costs for long sequences. To address this issue, linear attention [20, 31] has been proposed as an efficient alternative, providing linear complexity while maintaining the modeling capabilities. Position encoding is widely adopted to capture the sequential order of items users interact with due to its ability to enhance the positional awareness of contextual vectors. As illustrated in Fig. 1(a) and (b), the above attention-based methods rely on learnable position encoding. The learned positional encoding requires more parameters than fixed positional encoding schemes since each position needs its own learned vector, and they do not have any inherent structure that captures the notion of relative distances between positions. To handle these drawbacks, some works [46, 50] employ an improved Rotary Position Encoding (RoPE), which injects relative positional information in a more efficient manner. However, they still use the dot-product attention mechanism, which limits its scalability for long sequences. Furthermore, the RoPE exhibits inefficiency in multi-head attention scenarios, restricting the model's overall expressive capability.

Linear attention, while effective for grasping overarching long-term dependencies, can struggle to identify ***Fine-Grained Local Preferences***. In recommendation contexts, user interactions often reveal dense, localized activity sequences. For instance, a user might intensely engage with a specific series of related items. They could watch multiple episodes of *The Avengers* and then immediately consume

*Corresponding author
jhwen@cqu.edu.cn (J. Wen)
ORCID(s):

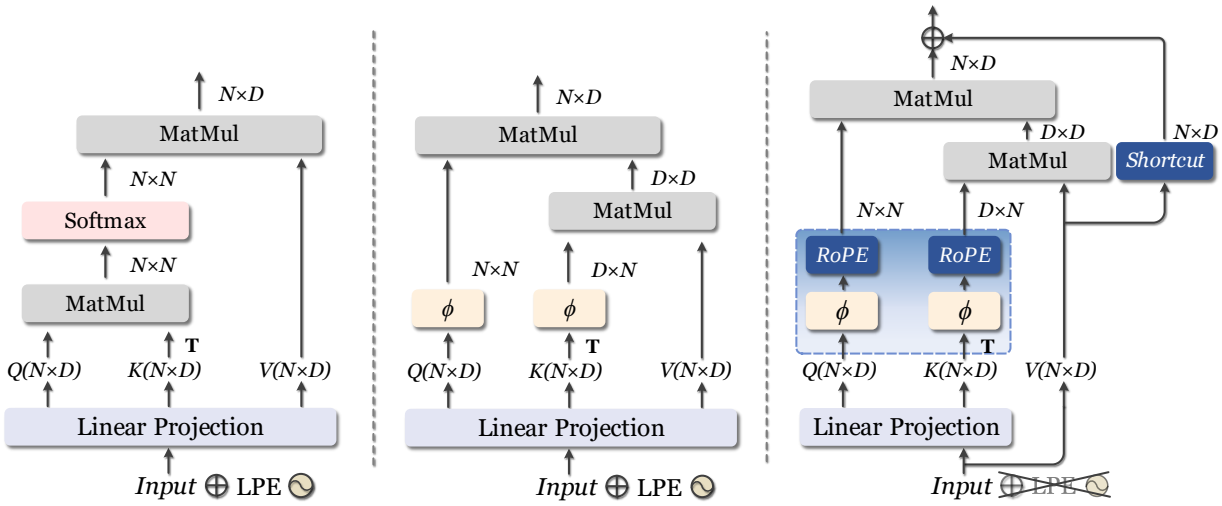

Figure 1: Illustration of differences of SASRec, LinRec, and our RecGRELA. ϕ denotes the kernel function.

supplementary content detailing Marvel's production process. Subsequently, they might delve into documentaries that explore the creators behind such cinematic universes. These closely related, short-term sequences of engagement are crucial for understanding immediate user intent, yet many existing approaches overlook their significance in long-term sequential recommendation.

Even when linear attention successfully captures both long-term interests and short-term local preferences, a significant challenge persists: ***when a user's preferences really change, it's hard for the model to identify them***. For instance, consider a user like historical documentaries over a weekend, binge-watches several interconnected Marvel movies like *The Avengers* series, and then explores some behind-the-scenes content about their production. A model that struggles to differentiate signals might misinterpret this intense but potentially short-lived burst of activity as a long-term shift in preference. Consequently, it might aggressively recommend a deluge of Marvel-related content for weeks – from other superhero films they've already seen, to comic book adaptations, or even tangentially related action movies. This happens because the model overweights fleeting interests. The user might simply want a weekend of superhero fun and is now looking for a historical drama, getting frustrated by the repetitive and misaligned suggestions.

To solve the aforementioned challenges, we propose **RecGRELA**, a novel long-term sequential **Rec**ommendation model with **G**ated **R**otary **E**nhanced **L**inear **A**ttention block (as shown in Fig. 1(c)) to effectively capture long-range dependency. Specifically, we design a Rotary Enhanced Linear Attention (RELA) module. RELA builds upon the efficiency of linear attention, directly addressing the quadratic complexity issue of standard Transformers, but significantly enhances its positional awareness by incorporating Rotary Position Encoding (RoPE). This allows the model to effectively capture relative inter-item positional relationships in a parameter-efficient manner, overcoming the drawbacks of purely learnable or fixed encodings and the inefficiency of RoPE in standard multi-head dot-product attention. Concurrently, it integrates local preferences efficiently via a Local Shortcut operation with causal 1D convolution, and we present theoretical insights based on matrix rank analysis. We further introduce a SiLU-based gated mechanism to identify whether user behavior is a temporary burst of interest or a true shift of long-term interest via adjusting the output of RELA. With this gated mechanism, we remove the linear projection on the value matrix of the attention module to reduce the computational cost.

The contributions of this work are summarized as follows:

- We propose a Rotary Enhanced Linear Attention (RELA) module that integrates Rotary Position Encoding (RoPE) into the linear attention mechanism to address efficiency issues in long-sequence modeling. By replacing dot-product attention with linear attention, our approach reduces computational costs while maintaining the ability to capture inter-item positional relationships.
- We develop a Local Shortcut operation to inject dense local interaction into the linear attention framework. This mechanism combines causal 1D convolution, explicitly modeling local preferences of user behaviors without sacrificing computational efficiency.

- We introduce a SiLU-based gated linear attention mechanism to distinguish true shifts in user preferences from transient local interests. Based on these designs, we further construct the RecGRELA, a novel long-term sequential recommendation paradigm that achieves competitive performance compared with state-of-the-art SRSs on four benchmark datasets, keeping low memory overhead.

2. PRELIMINARY

2.1. Problem Definition

Considering a set of users $\mathcal{U} = \{u_i\}_{i=1}^{|\mathcal{U}|}$ and a set of items $\mathcal{V} = \{v_i\}_{i=1}^{|\mathcal{V}|}$. We use $\mathcal{S}_u = \{v_t\}_{t=1}^{T}$ to denote an interaction sequence of T chronologically ordered items that the user u has interacted with. Given a user interaction sequence $\mathcal{S}_u$, the task of sequential recommendation is to compute a ranking list consisting of Top-k items that the user u is likely to interact with at the next time step $T+1$. We define N as the sequential length and D as its corresponding dimension. Typically, the N is often relatively short, but in some complex scenarios, where the N satisfies $N > 1.5D$ (e.g., when D=64, the $N > 96$), the sequence is regarded as "**long-term**" [31].

2.2. Dot-Product Attention and Linear Attention

Let $\boldsymbol{X} \in \mathbb{R}^{N\times D}$ denotes an input sequence. Formally, the input sequence $\boldsymbol{X}$ is projected by three projection matrices $\boldsymbol{W}_Q, \boldsymbol{W}_K, \boldsymbol{W}_V \in \mathbb{R}^{D\times d}$, to corresponding representations $\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q$, $\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K$, and $\boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V$. The **dot-product attention** [52] can be written as:

$$\boldsymbol{O}_m = \sum_{n=1}^{N} \frac{\exp\left(\boldsymbol{Q}_m \boldsymbol{K}_n^\top / \sqrt{d}\right)}{\sum_{n=1}^{N} \exp\left(\boldsymbol{Q}_m \boldsymbol{K}_n^\top / \sqrt{d}\right)} \boldsymbol{V}_n, \tag{1}$$

where $\boldsymbol{Q}_m, \boldsymbol{K}_n, \boldsymbol{V}_n \in \mathbb{R}^{1\times d}$, and d is head dimension. In this case, calculating attention weights in dot-product attention leads to the computation complexity of $\mathcal{O}(N^2)$ with respect to the input sequence length N. For autoregressive modeling, dot-product attention can be implemented by incorporating a masking mechanism, which restricts the influence on the m-th position to a position n satisfying $n \leq m$.

Linear attention [20] is proposed as an effective alternative that reduces the computation complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ compared with dot-product attention. Specifically, linear attention modifies the dot-product attention by replacing the non-linear Softmax operation with linear normalization, while incorporating an additional kernel function ϕ in query matrix $\boldsymbol{Q}$ and key matrix $\boldsymbol{K}$:

$$\boldsymbol{O}_m = \sum_{n=1}^{N} \frac{\phi\left(\boldsymbol{Q}_m\right)\phi\left(\boldsymbol{K}_n\right)^\top}{\sum_{n=1}^{N} \phi\left(\boldsymbol{Q}_m\right)\phi\left(\boldsymbol{K}_n\right)^\top} \boldsymbol{V}_n. \tag{2}$$

By utilizing the associative property of matrix multiplication, the computation is reordered from $(\boldsymbol{Q}\boldsymbol{K}^\top)\boldsymbol{V}$ to $\boldsymbol{Q}(\boldsymbol{K}^\top\boldsymbol{V})$:

$$\boldsymbol{O}_m = \frac{\phi\left(\boldsymbol{Q}_m\right)\left(\sum_{n=1}^{N} \phi\left(\boldsymbol{K}_n\right)^\top \boldsymbol{V}_n\right)}{\phi(\boldsymbol{Q}_m)\left(\sum_{n=1}^{N} \phi\left(\boldsymbol{K}_n\right)^\top\right)}, \tag{3}$$

where the computational complexity is decreased to $\mathcal{O}(N)$. In [20], the kernel function employs the Exponential Linear Unit (ELU) activation [2], which prevents gradient vanishing when processing negative values of $\boldsymbol{X}$. In our work, we follow this setting for a fair comparison.

2.3. Position Encoding

Absolute Position Encoding (**APE**) [52] incorporates fixed values into the embeddings of sequence elements, establishing a consistent reference frame across all tokens. For a token at the position t within a sequence of length N, this encoding is computed through a deterministic function utilizing paired sine and cosine operations across different frequencies. The formulation is denoted as follows:

$$\begin{aligned} PE\left(p, 2t\right) &= \sin\left(p/10000^{2t/(D/2)}\right), \\ PE\left(p, 2t+1\right) &= \cos\left(p/10000^{2t/(D/2)}\right), \end{aligned} \tag{4}$$

where p is the position of the item, t is the dimension of p.

Learnable Position Encoding (**LPE**) [53, 19] adopts a distinct methodology by modeling positional information as learnable parameters that are optimized during the training process. Specifically, for an item embedding $\boldsymbol{x}$ at position m, the transformation is performed as follows:

$$\boldsymbol{x}_m = \boldsymbol{x} + p_m, \tag{5}$$

where p_i denotes the APE or LPE.

Rotary Position Encoding (**RoPE**) [46] captures relative positional information [43] through absolute position encoding methods via encoding the relative positional information with a rotation matrix $\mathbf{R} \in \mathbb{C}^{N\times(d/2)}$:

$$\mathbf{R}\left(m, t\right) = e^{i\theta_t m}, \tag{6}$$

where i denotes the imaginary unit in complex numbers. Following [52], the θ_t is set to $10000^{-t/(d/2)}$, where $t \in [1, 2, ..., d/2]$. Then RoPE converts $\boldsymbol{Q}_m$, $\boldsymbol{K}_n \in \mathbb{R}^{N\times d}$ to complex vector $\bar{\boldsymbol{Q}}_m$, $\bar{\boldsymbol{K}}_n \in \mathbb{C}^{N\times(d/2)}$ by considering $(2t)$-th dim as real part and $(2t+1)$-th dim as imaginary part. The RoPE is applied to complex vectors of query and key as:

$$\begin{aligned} \bar{\boldsymbol{Q}}'_m &= \bar{\boldsymbol{Q}}_m \odot \mathbf{R}\left(m, t\right), \\ \bar{\boldsymbol{K}}'_n &= \bar{\boldsymbol{K}}_n \odot \mathbf{R}\left(n, t\right), \\ \boldsymbol{A}'_{(m,n)} &= \mathrm{Re}[\bar{\boldsymbol{Q}}'_m \bar{\boldsymbol{K}}'_n] = \mathrm{Re}[\bar{\boldsymbol{Q}}_m \bar{\boldsymbol{K}}^*_n e^{i(m-n)\theta_t}], \end{aligned} \tag{7}$$

where $\mathrm{Re}[\cdot]$ represents real part of the complex vector and $*$ means complex conjugates, and $\odot$ denotes Hadamard product. Note that the attention matrix $\boldsymbol{A}_{(m,n)}$ with RoPE implies relative position in rotation form $e^{i(m-n)\theta_t}$ for $(D/2)$ frequencies. We can define the RoPE operation $\mathcal{R}\left(\cdot\right)$ to an input real vector $\boldsymbol{X}$ as follows:

$$\mathcal{R}\left(\boldsymbol{X}\right) = \mathcal{C}^{-1}\left(\mathcal{C}\left(\boldsymbol{X}\right) \odot \mathbf{R}\right), \tag{8}$$

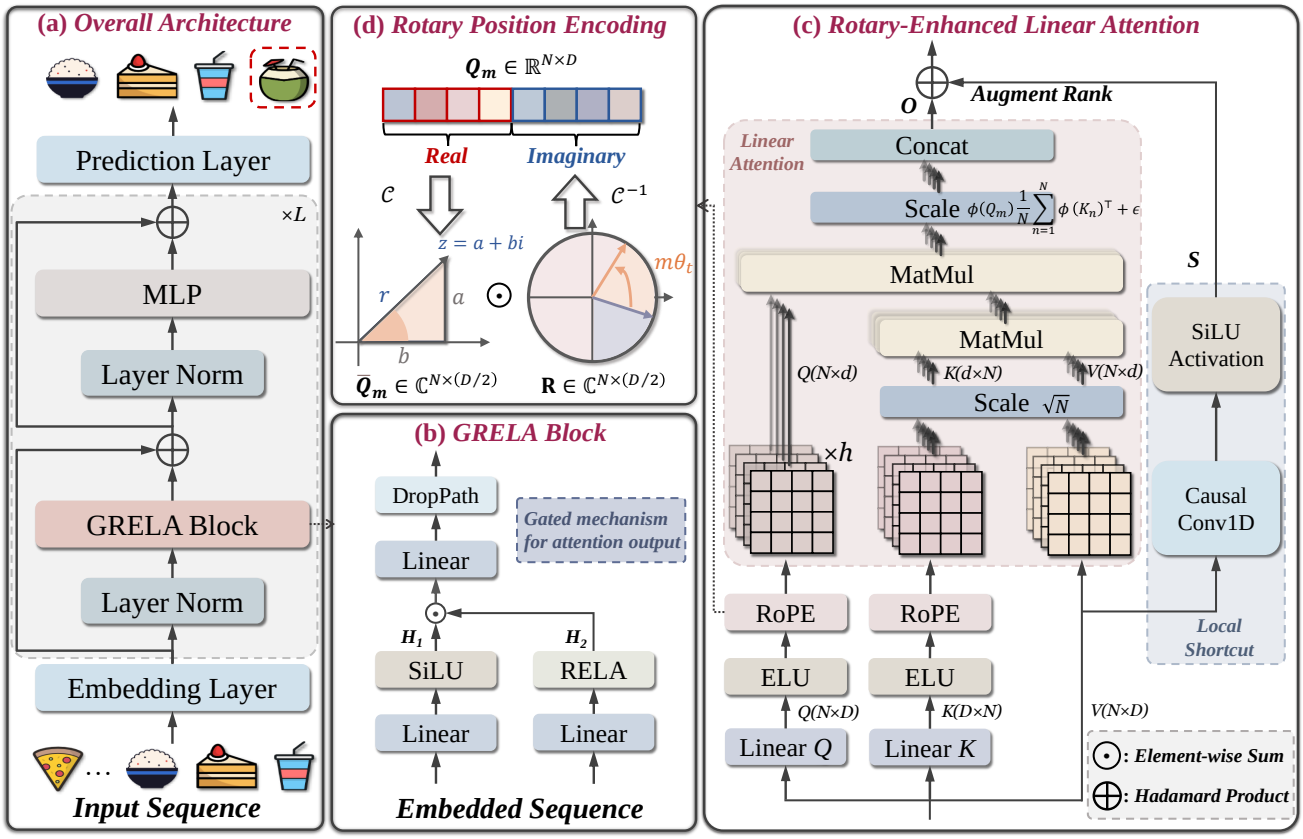

Figure 2: The overview of our proposed architecture. The Overall Architecture (a) processes an input sequence through an Embedding Layer, which is then fed into a stack of L identical encoder layers. Each encoder layer consists of Layer Normalization, a GRELA Block, another Layer Normalization, and an MLP, finally leading to a Prediction Layer. The central components are the GRELA Block (b), which utilizes RoPE (Rotary Position Encoding, d), and the RELA module (c). Within the RELA module, query (Q) and key (K) are processed by ELU activation and the RoPE. Attention scores are computed via matrix multiplication of the RoPE-enhanced Q and K (after K is scaled), followed by another scaling and a concatenation step. A Causal Conv1D and SiLU activation processes the value (V) projection to capture local context.

where $\mathcal{C}(\cdot)$ converts a real-valued tensor into a complex-valued tensor, and $\mathcal{C}^{-1}(\cdot)$ converts a complex-valued tensor into a real-valued tensor.

3. METHODOLOGY

In this section, we introduce our proposed framework, RecGRELA. The framework overview is shown in Figure 2. The presentation is structured as follows: initially, we introduce the embedding layer; secondly, we elaborate on the core components of the GRELA Block; subsequently, we present the details of the Multi-Layer Perceptron, and finally, we describe the prediction layer and training strategies.

3.1. Embedding Layer

Following [19], we utilize an embedding layer to map the training sequence $\mathcal{S}_u$ into a high-dimensional space. The embedding layer uses a learnable embedding matrix $\boldsymbol{E} \in \mathbb{R}^{|\mathcal{V}|\times D}$, where $\mathcal{V}$ denotes the item set and D represents the embedding dimension. Given an input interaction sequence $\mathcal{S}_u$ of length N, it can be projected into the initial embedding matrix $\boldsymbol{H}_o \in \mathbb{R}^{N\times D}$ as follows:

$$\boldsymbol{H}_o = \boldsymbol{E}\mathcal{S}_u. \tag{9}$$

Then we apply the Dropout [45] and layer normalization [23] operations to obtain the final embedding $\boldsymbol{H} \in \mathbb{R}^{N\times D}$:

$$\boldsymbol{H} = \text{LayerNorm}\left(\text{Dropout}\left(\boldsymbol{H}_o\right)\right). \tag{10}$$

Attention-based sequential recommendation models [19, 47, 26, 31] typically inject a learnable position encoding into the final embedding, while our method captures position awareness with rotary position encoding by multiplying the rotation matrix with outputs of the kernel function in linear attention. This design eliminates the need to incorporate positional information into the embedding layers.

3.2. GRELA Block

In this part, we present the design of our proposed GRELA Block, which includes the Rotary-Enhanced Linear Attention module and the Local Shortcut operation.

3.2.1. Rotary-Enhanced Linear Attention

Overall Design. Previous work [31] provides $\mathcal{O}(N)$ computational complexity of attention mechanism to alleviate the issue of quadratic complexity in dot-product attention for sequential recommendation. However, it ignores the high cost of learnable position encoding in long-sequence modeling. Meanwhile, some works [50, 34] notice these issues and propose improved RoPE to address them, but unfortunately, they fail to address the problem of dot-product attention with $\mathcal{O}(N^2)$ complexity. To bridge this gap, we propose a novel Rotary-Enhanced Linear Attention (RELA) mechanism, aiming to simultaneously resolve the issues of quadratic complexity and ineffective position encoding in long-sequence modeling. The motivation stems from the observation that other position encoding methods operate directly on the input sequence or the attention matrix. However, linear attention does not compute the attention matrix, which means other methods can only modify the input sequence. In contrast, RoPE achieves relative position encoding by leveraging absolute position encoding, by applying transformations only to the queries $\boldsymbol{Q}$ and keys $\boldsymbol{K}$. This design makes RoPE suitable for integration with linear attention. In a general situation, following Eq.(7), we revise Eq. (3) as follows:

$$\boldsymbol{O}_m = \frac{\mathcal{R}\left(\phi\left(\boldsymbol{Q}_m\right)\right)\left(\mathcal{R}\left(\sum_{n=1}^{N}\phi\left(\boldsymbol{K}_n\right)^\top\right)\boldsymbol{V}_n\right)}{\phi(\boldsymbol{Q}_m)\left(\sum_{n=1}^{N}\phi\left(\boldsymbol{K}_n\right)^\top\right)}, \quad (11)$$

where $\mathcal{R}(\cdot)$ denotes injecting the RoPE into queries or keys. Following [20], we set $\phi(x) = \mathrm{ELU}(x) + 1$. This kernel function ensures that the denominator remains positive to avoid the risk of division by zero, while the summation in the numerator also contains no negative terms.

Inspired by techniques for stabilizing gradients in dot-product attention [52], we further incorporate scaling factors. Specifically, the keys $\boldsymbol{K}$ and values $\boldsymbol{V}$ are scaled by $\sqrt{N}$ (where N denotes the sequence length). This emulates the scaling mechanism used in conventional dot-product attention and helps regulate the dynamic range of the resulting key-value memory. Furthermore, the normalization term in the denominator, which is critical for linear attention, is constructed using the dot product between the query $\phi(\boldsymbol{Q}_m)$ and the mean of keys $\phi(\boldsymbol{K}_n)$. A small constant ϵ is added to this denominator for numerical stability, preventing division by zero and ensuring well-behaved gradients. This approach enables more fine-grained control over the attention distribution. The final RELA is formulated as follows:

$$\boldsymbol{O}_m = \frac{\mathcal{R}\left(\phi\left(\boldsymbol{Q}_m\right)\right)\left(\left(\mathcal{R}\left(\sum_{n=1}^{N}\phi\left(\boldsymbol{K}_n\right)^\top\right)\boldsymbol{V}_n/\sqrt{N}\right)\right)}{\phi(\boldsymbol{Q}_m)\frac{1}{N}\sum_{n=1}^{N}\phi\left(\boldsymbol{K}_n\right)^\top + \epsilon}. \quad (12)$$

In-Depth Analysis for Rotaty Position Encoding. To intuitively understand why Eq.(7) embodies the concept of *Rotation*, we introduce RoPE with the head dimension $d = 2$. When the dimension $d = 2$, we take $\boldsymbol{Q}_m$ as an example for a detailed derivation. We first express it in its complex form:

$$\boldsymbol{Q}_m = \left[\boldsymbol{Q}_m^{(1)}, \boldsymbol{Q}_m^{(2)}\right] = \left[\boldsymbol{Q}_m^{(1)} + i\boldsymbol{Q}_m^{(2)}\right], \quad (13)$$

where $\boldsymbol{Q}_m^{(1)}$ and $\boldsymbol{Q}_m^{(2)}$ indicate the components of $\boldsymbol{Q}_m$ expressed in the 2D coordinates. Based on Euler's formula, $e^{im\theta}$ can be rewritten as follows:

$$e^{im\theta} = \cos m\theta + i\sin m\theta. \quad (14)$$

Then, we have:

$$\begin{aligned}\boldsymbol{Q}_m e^{im\theta} &= \left(\boldsymbol{Q}_m^{(1)} + i\boldsymbol{Q}_m^{(2)}\right)(\cos(m\theta) + i\sin(m\theta)) \\ &= \left(\boldsymbol{Q}_m^{(1)}\cos(m\theta) - \boldsymbol{Q}_m^{(2)}\sin(m\theta)\right) + i\left(\boldsymbol{Q}_m^{(2)}\cos(m\theta) + \boldsymbol{Q}_m^{(1)}\sin(m\theta)\right).\end{aligned} \quad (15)$$

Then, the result is re-expressed in the form of a real-valued vector:

$$\begin{aligned}\boldsymbol{Q}_m e^{im\theta} &= [\boldsymbol{Q}_m^{(1)}\cos(m\theta) - \boldsymbol{Q}_m^{(2)}\sin(m\theta), \\ &\quad \boldsymbol{Q}_m^{(2)}\cos(m\theta) + \boldsymbol{Q}_m^{(1)}\sin(m\theta)] \\ &= \begin{pmatrix}\cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta\end{pmatrix}\begin{pmatrix}\boldsymbol{Q}_m^{(1)} \\ \boldsymbol{Q}_m^{(2)}\end{pmatrix}.\end{aligned} \quad (16)$$

Evidently, the matrix $\begin{pmatrix}\cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta\end{pmatrix}$ describes a rotational transformation applied to the embedded sequence.

3.2.2. Local Shortcut and Gated Mechanism

Attention mechanisms excel at long-range modeling, yet local behavior modeling is equally vital for long-term sequential recommendation systems, as user interactions often display strong local preferences (e.g., session-based behaviors) crucial for next-item prediction. However, linear attention variants falter here. Unlike Softmax attention's sharp focus, linear attention's smooth distribution tends to average features, obscuring crucial local cues [13]. Moreover, RoPE suffers from long-term decay [46], meaning the attention score decreases as the relative distance increases. Because of linear attention's smoothing and RoPE's distance decay, precisely modeling crucial local patterns for long-term sequential recommendation becomes difficult. To address these issues, we introduce a Local Shortcut to mitigate the long-term decay of RELA. Specifically, we use a causal 1D convolution [10] followed by a SiLU activation [41] to capture local preferences in the user interaction sequence:

$$\boldsymbol{S} = \mathrm{SiLU}\left(\mathrm{CausalConv1d}\left(\boldsymbol{V}\right)\right), \quad (17)$$

where $\boldsymbol{V}$ denotes the input sequence, and $\boldsymbol{S}$ represents the output of the temporal convolution. Then $\boldsymbol{S}$ is connected with the output $\boldsymbol{O}$ of linear attention as a shortcut to form the final output $\boldsymbol{O}_f$ of RELA:

$$\boldsymbol{O}_f = \boldsymbol{O} + \boldsymbol{S}. \quad (18)$$

With the local shortcut, each query only focuses on several adjacent items instead of the whole sequence. This locality ensures that even if the linear attention values corresponding to two queries are the same, we can still get different outputs from different local preferences.

The theoretical motivation behind such a design concerns the upper bound on the rank of the attention matrix in linear attention [13]. The rank of the attention matrix is bounded by the length of the sequence N and the feature dimension d for each head:

$$\begin{aligned}\operatorname{rank}(\phi(\boldsymbol{Q})\phi(\boldsymbol{K})^{\top}) &\leq \min\{\operatorname{rank}(\phi(\boldsymbol{Q})), \operatorname{rank}(\phi(\boldsymbol{K}))\} \\ &\leq \min\{N, d\},\end{aligned} \tag{19}$$

where d is often smaller than N in long-term sequence scenarios. Under these conditions, the upper bound on the rank of the attention matrix is constrained to remain at a low level. The rank represents the number of linearly independent vectors, i.e., the dimensionality, contained in the matrix. When an N-dimensional attention matrix has a rank of d, it effectively operates as a d-dimensional matrix in terms of its representational capacity, which fundamentally limits the expressive power of the attention mechanism. Referring back to Eq. 18, the operation on the attention matrix can be reformulated as:

$$\begin{aligned}\boldsymbol{O}_f &= \phi(\boldsymbol{Q})\phi(\boldsymbol{K}) + \operatorname{CSC}(\boldsymbol{V}) \\ &= (\phi(\boldsymbol{Q})\phi(\boldsymbol{K}) + \boldsymbol{M}_{\mathrm{CSC}})\boldsymbol{V} = \boldsymbol{M}_{eq}\boldsymbol{V},\end{aligned} \tag{20}$$

where we omit the normalization term for simplicity, $\operatorname{CSC}(\boldsymbol{V})$ denotes the causal convolution operation on $\boldsymbol{V}$. Let $\boldsymbol{M}_{\mathrm{CSC}}$ represent the sparse matrix associated with the causal convolution and non-linear activation function. $\boldsymbol{M}_{eq}$ denotes the equivalent full attention map. Since M_{CSC} has the potential to be a full-rank matrix, which increases the upper bound of the rank of the equivalent attention matrix. To this end, we only use a little computational overhead to improve the performance of RELA.

To address the challenge of differentiating between temporary local preferences and genuine shifts in long-term user interests, we design a gated mechanism to intelligently modulate RELA's output. We utilize a SiLU gate [39, 12] and functioning as an *output gate*, learns to derive weights from the interaction sequence. These weights then adjust the influence of different signals within the user behavior sequence processed by RELA. This allows the model to discern whether a new burst of user behaviors represents a strong but potentially transient local preference or signifies a true evolution in the user's underlying taste. The overall structure of the GRELA Block is shown as follows:

$$\begin{aligned}\boldsymbol{H}_g &= \operatorname{LayerNorm}(\boldsymbol{H}), \\ \boldsymbol{H}_1 &= \operatorname{SiLU}\left(\boldsymbol{W}_1\boldsymbol{H}_g + \boldsymbol{b}_1\right), \\ \boldsymbol{H}_2 &= \operatorname{RELA}\left(\boldsymbol{W}_2\boldsymbol{H}_g + \boldsymbol{b}_2\right),\end{aligned} \tag{21}$$

where $\boldsymbol{H}_g$ is the input of the GRELA block, i.e., when $L = 1$, the input $\boldsymbol{H}$ is from the embedding layer, $\boldsymbol{W}_1, \boldsymbol{W}_2 \in \mathbb{R}^{D\times D}$ are the weight matrices of the linear projection, $\boldsymbol{b}_1$, $\boldsymbol{b}_2$ are the biases. Then a Hadamard product is performed on $\boldsymbol{H}_1$ and $\boldsymbol{H}_2$ followed by a linear projection and Droppath [22] operation:

$$\boldsymbol{H}_3 = \operatorname{DropPath}\left(\boldsymbol{W}_3\left(\boldsymbol{H}_1 \odot \boldsymbol{H}_2\right) + \boldsymbol{b}_3\right) + \boldsymbol{H}, \tag{22}$$

where $\boldsymbol{W}_3 \in \mathbb{R}^{D\times D}$ is the weight matrix, $\boldsymbol{b}_3$ is the bias.

Meanwhile, due to the linear projection before the RELA, we can treat it as an *input gate* of linear attention like long-term memory network (LSTM), previous studies [11, 51] have demonstrated that the input gate plays a less significant role compared to the forget gate in LSTM. Inspired by this, we **remove the linear projection for the value matrix** of linear attention. To this end, we only use the weight matrices on query and key, reducing training costs.

3.3. Multi-Layer Perceptron

To capture the complex features, we further leverage a Multi-Layer Perceptron (MLP) [19]:

$$\begin{aligned}\boldsymbol{H}_4 &= \operatorname{LayerNorm}\left(\boldsymbol{H}_3\right), \\ \boldsymbol{H}_L &= \operatorname{Drop}\left(\operatorname{Drop}\left(\operatorname{GELU}\left(\boldsymbol{H}_4\boldsymbol{W}_4 + \boldsymbol{b}_4\right)\right)\boldsymbol{W}_5 + \boldsymbol{b}_5\right) + \boldsymbol{H}_3,\end{aligned} \tag{23}$$

where $\boldsymbol{W}_4 \in \mathbb{R}^{D\times 4D}, \boldsymbol{W}_5 \in \mathbb{R}^{4D\times D}$ denote the weight matrices, $\boldsymbol{b}_4$, $\boldsymbol{b}_5$ are also the biases. $\boldsymbol{H}_L$ indicates the final output representation, and $\boldsymbol{H}_4$ denotes the output of layer normalization after the GRELA block. To stabilize the training process and enhance gradient flow through the network, we employ a residual connection [16]. For generality, the subscript (0) indicates that the final user representation is obtained using a single GRELA layer.

3.4. Prediction Layer and Training

After the computation of L GRELA blocks and the MLP, we get the final representation of behavior sequences with user interest. Based on this, we compute the dot product between it and the embeddings of all items $\boldsymbol{E} \in \mathbb{R}^{|\mathcal{V}|\times D}$. This result is then scaled using the Softmax function to obtain the interaction probability score:

$$\hat{\mathcal{Y}} = \operatorname{Softmax}(\boldsymbol{E}^{\top}\boldsymbol{H}_L). \tag{24}$$

Hence, following previous works [19, 30], we expect to minimize the cross-entropy between the predicted recommendation results $\hat{\mathcal{Y}}$, and the ground truth items $\mathcal{Y}$:

$$\mathcal{L}_{CE}\left(\mathcal{Y}, \hat{\mathcal{Y}}\right) = \mathcal{Y}\log\left(\hat{\mathcal{Y}}\right) + (1-\mathcal{Y})\left(1 - \log\left(\hat{\mathcal{Y}}\right)\right). \tag{25}$$

3.5. Complexity Analysis

We analyze the computational complexity of RELA (as shown in Eq. (12)), focusing on several key components. To begin with, the kernel function in matrices $\boldsymbol{Q}, \boldsymbol{K}, \in \mathbb{R}^{N\times d}$, incurs a computational cost of $\mathcal{O}(Nd)$. Subsequently, the RoPE operator $\mathcal{R}(\cdot)$ contributes a time complexity that is also $\mathcal{O}(Nd)$. Furthermore, the matrix multiplication operation has a $\mathcal{O}(Nd^2)$ complexity. This contrasts with the original dot-product attention mechanism, which exhibits a complexity of $\mathcal{O}(N^2d)$. Consequently, the overall time complexity of the RELA is $\mathcal{O}(Nd^2)$.

Table 1
Statistics of the experimental datasets.

Datasets	#Users	#Items	#Interactions	Avg. UA	Avg. IA	Sparsity
ML-1M	6,041	3,417	999,611	165.5	292.6	95.15%
ML-20M	138,494	18,346	19,984,024	144.3	1,098.3	99.21%
ML-32M	200,949	43,885	31,921,467	158.9	727.4	99.63%
LFM-1B	78,113	179,515	15,628,182	200.1	87.1	99.89%
Netflix	120,378	16,087	21,767,054	180.8	1,353.1	98.86%

4. Experiments

4.1. Experimental Settings

4.1.1. Datasets

We conduct experiments and verify the effectiveness of our approach on five datasets, including MovieLens-1M, 20M, and 32M [1] (dubbed as ML-1M, ML-20M, ML-32M), LFM-1B [2], and Netflix [3]. The dataset statistics are shown in Table 1, where *Avg. UA* and *Avg. IA* denote the average actions of users and items. Notably, for MovieLens datasets, we filter out users and items with fewer than 5 interactions. For LFM-1B datasets, we use the processed atomic files of LFM1B-Artists (merged) and filter out users with fewer than 50 and more than 500 interactions. Netflix is a huge-scale dataset, containing approximately 100 million interactions and five thousand *Avg. IA*. To simplify the training process, we filter out users with fewer than 50 interactions while retaining items with interactions ranging from 50 to 15,000 for this dataset.

4.1.2. Baseline Models

To verify the effectiveness and efficiency of RecGRELA, we make comparisons between RecGRELA and strong RNN-, Attention-, and Mamba-based baselines: **LinRec** [31]: This model reduces the computational costs via linear attention in transformer-based models. We choose SASRec [19] as the backbone of LinRec.**LRURec** [62]: This method is based on linear recurrent units [36] for sequential recommendation. **Mamba4Rec** [30]: This is the first Mamba-based method, which explores applying the Mamba architecture in sequential recommendation tasks. **EchoMamba4Rec** [56]: This method processes sequences both forward and backward to enhance the sequential representation with Mamba and frequency domain filter. **EulerFormer** [50]: This method proposes a transformer variant with complex vectors based on Euler's formula and a contrastive learning method to improve contextual representations. **RecBLR** [29]: This model shows a behavior-dependent linear recurrent unit to enhance user behavior modeling with a hardware-aware parallel scan approach. **SIGMA** [32]: This model employs a flipped Mamba to build a bidirectional framework to improve short sequence representations and contextual modeling.

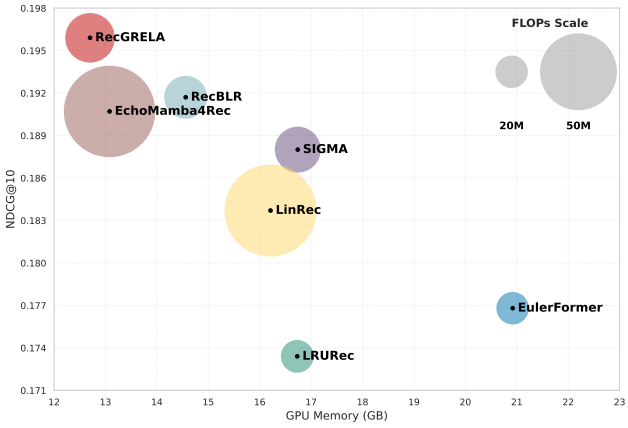

Figure 3: RecGRELA vs. baselines in terms of the GPU memory and FLOPs of the training stage on the ML-1M dataset.

4.1.3. Evaluation Protocols

We evaluate performance using three standard metrics: *Hit Rate* (HR), *Normalized Discounted Cumulative Gain* (NDCG), and *Mean Reciprocal Rank* (MRR). These metrics are assessed based on top-K recommendation lists, specifically using cutoff values $K = 5$ and $K = 10$, denoted as HR@5, NDCG@5, MRR@5, HR@10, NDCG@10, and MRR@10, respectively.

4.1.4. Implementation Details

We utilize the RecBole library [4] [66, 59], a unified framework for implementing sequential recommendations. To ensure a fair comparison, we fix the embedding size to 64, the hidden size of the MLP to 256, and the training batch size to 2048 for all the models. We directly adopt the values reported in their original manuscripts for the remaining hyperparameters of the baseline models. The number of blocks L is set to 4. The number of attention heads h is set to 4. The kernel size for 1D causal convolution is 4. We set the Dropout and Droppath rate to 0.1. We use Adam [21] with a learning rate of 0.001 for optimizing our model. The maximum sequence length N is set to 200 for all the datasets.

[1] https://grouplens.org/datasets/movielens/
[2] https://www.cp.jku.at/datasets/LFM-1b/
[3] https://www.kaggle.com/datasets/netflix-inc/netflix-prize-data
[4] https://github.com/RUCAIBox/RecBole

Table 2
Performance comparisons on ML-1M, ML-20M, ML-32M, and Netflix datasets. Best and second-best results per row are shown in **bold** and underlined format, respectively. "Improv." denotes the percentage improvement of the best-performing model over the second-best for each metric. The "$\star$" denotes the statistical significance ($p < 0.05$) of the results of MegaRec compared to the best baseline.

Dataset	Metric	LinRec	LRURec	Mamba4Rec	EchoMamba	EulerFormer	RecBLR	SIGMA	RecGRELA	Improv.
ML-1M	HR@5	0.2187	0.2134	0.2211	0.2243	0.2159	0.2293	0.2325	**0.2364**$\star$	1.68%
	NDCG@5	0.1472	0.1418	0.1521	0.1539	0.1493	0.1604	0.1587	**0.1653**$\star$	3.05%
	MRR@5	0.1263	0.1217	0.1314	0.1328	0.1281	0.1377	0.1345	**0.1419**$\star$	3.05%
	HR@10	0.3109	0.3026	0.3230	0.3253	0.3089	0.3258	0.3291	**0.3311**$\star$	0.61%
	NDCG@10	0.1837	0.1734	0.1869	0.1907	0.1768	0.1917	0.1898	**0.1959**$\star$	2.19%
	MRR@10	0.1448	0.1340	0.1453	0.1496	0.1365	0.1507	0.1472	**0.1545**$\star$	2.52%
ML-20M	HR@5	0.1886	0.1719	0.1813	0.1837	0.1782	0.1890	0.1945	**0.2007**	3.19%
	NDCG@5	0.1297	0.1146	0.1248	0.1271	0.1219	0.1316	0.1361	**0.1415**	3.97%
	MRR@5	0.1103	0.0941	0.1047	0.1072	0.1015	0.1127	0.1169	**0.1220**	4.36%
	HR@10	0.2685	0.2572	0.2649	0.2679	0.2482	0.2664	0.2729	**0.2791**	2.27%
	NDCG@10	0.1554	0.1488	0.1537	0.1578	0.1432	0.1559	0.1613	**0.1667**	3.35%
	MRR@10	0.1209	0.1153	0.1222	0.1247	0.1112	0.1222	0.1272	**0.1324**	4.09%
ML-32M	HR@5	0.1568	0.1513	0.1621	0.1649	0.1587	0.1671	0.1698	**0.1802**$\star$	6.12%
	NDCG@5	0.1034	0.0989	0.1092	0.1117	0.1056	0.1151	0.1180	**0.1258**$\star$	6.61%
	MRR@5	0.0871	0.0826	0.0923	0.0948	0.0894	0.0980	0.1009	**0.1080**$\star$	7.04%
	HR@10	0.2397	0.2235	0.2372	0.2401	0.2208	0.2378	0.2426	**0.2519**$\star$	3.83%
	NDCG@10	0.1402	0.1296	0.1369	0.1398	0.1270	0.1379	0.1423	**0.1490**$\star$	4.71%
	MRR@10	0.1090	0.1015	0.1062	0.1083	0.1003	0.1073	0.1116	**0.1175**$\star$	5.29%
LFM-1B	HR@5	0.0755	0.0732	0.0768	0.0781	0.0745	0.0778	0.0834	**0.0860**$\star$	3.12%
	NDCG@5	0.0421	0.0401	0.0435	0.0448	0.0410	0.0447	0.0512	**0.0531**$\star$	3.71%
	MRR@5	0.0315	0.0298	0.0322	0.0339	0.0305	0.0339	0.0407	**0.0424**$\star$	4.18%
	HR@10	0.1345	0.1288	0.1352	0.1380	0.1333	0.1420	0.1421	**0.1460**$\star$	2.74%
	NDCG@10	0.0632	0.0603	0.0640	0.0655	0.0621	0.0702	0.0700	**0.0724**$\star$	3.13%
	MRR@10	0.0465	0.0441	0.0473	0.0481	0.0456	0.0482	0.0484	**0.0502**$\star$	3.72%
Netflix	HR@5	0.0900	0.0712	0.0809	0.0827	0.0781	0.0873	0.0897	**0.0922**$\star$	2.44%
	NDCG@5	0.0606	0.0443	0.0538	0.0559	0.0512	0.0581	0.0604	**0.0623**$\star$	2.81%
	MRR@5	0.0507	0.0369	0.0446	0.0467	0.0431	0.0483	0.0508	**0.0525**$\star$	3.35%
	HR@10	0.1368	0.1196	0.1305	0.1327	0.1191	0.1338	0.1359	**0.1386**$\star$	1.32%
	NDCG@10	0.0765	0.0650	0.0728	0.0736	0.0648	0.0747	0.0763	**0.0792**$\star$	3.53%
	MRR@10	0.0582	0.0484	0.0532	0.0553	0.0495	0.0567	0.0582	**0.0611**$\star$	4.98%

The early stopping epochs are 20 for ML-1M and 10 for other datasets. All the experiments are conducted on a single NVIDIA RTX4090.

4.2. Overall Performance

Table 2 reports the recommendation performance of RecGRELA compared to seven baselines on five datasets. From the table, we find several observations:

- Our RecGRELA outperforms all competing transformer-based, RNN-based, and Mamba-based baselines, with improvements ranging from 0.61% to 4.98%. Such a comparison highlights the effectiveness of our design in the long-term sequential recommendation.
- The performance improvements are more significant on larger datasets with richer interactions and more average action of items. For example, the Netflix dataset has almost 22M interactions and relatively long item actions (1.3K). Conversely, performance gains are more modest for sparser datasets with fewer interactions, users, and items, such as ML-1M. This phenomenon can be attributed to the data-rich environments providing sufficient information for the model to learn more complex patterns, leading to more accurate recommendation performance.
- The improvements in the metrics of NDCG and MRR are typically greater than those in HR. This discrepancy stems from their distinct evaluation criteria: NDCG and MRR employ position-sensitive weighting schemes prioritizing ranking precision, whereas HR operates as a binary relevance indicator. This

Table 3
Sensitivity analysis on different numbers of layers. The sequence length is set to 200. Time indicates the training time per epoch, and Memory indicates the GPU memory.

#Layers	HR@5	NDCG@5	MRR@5	HR@10	NDCG@10	MRR@10	Time	Memory
1	0.1512	0.0987	0.0815	0.2874	0.1669	0.1301	20.70s	5.96GB
2	0.1699	0.1050	0.0899	0.3184	0.1838	0.1426	39.49s	7.71GB
3	0.1753	0.1088	0.0921	0.3228	0.1866	0.1450	58.00s	10.16GB
4	**0.1865**	**0.1152**	**0.0988**	**0.3311**	**0.1959**	**0.1545**	76.50s	12.70GB
5	0.1801	0.1115	0.0954	0.3280	0.1916	0.1499	94.88s	15.14GB
6	0.1776	0.1109	0.0948	0.3242	0.1912	0.1506	113.43s	17.68GB

ranking superiority enables RecGRELA to generate recommendations with enhanced positional accuracy, elevating user-preferred items to top-tier placements while maintaining contextual relevance. This can also be attributed to introducing RoPE, which enhances the model's ability to capture positional information.

4.3. Efficiency Analysis

We compare the training efficiency of our RecGRELA with state-of-the-art sequential recommendation models in terms of GPU memory and FLOPs. The results are shown in Fig. 3. The results are based on the ML-1M dataset with a sequence length of 200. Our RecGRELA demonstrates superior efficiency and performance, achieving the highest NDCG@10 of 0.1959 with the lowest GPU memory (12.7GB) among the compared models.

Specifically, our proposed RecGRELA achieves a 2% performance improvement while requiring only 87% GPU memory consumption and comparable FLOPs compared to RecBLR. More significantly, when compared with LinRec, our method demonstrates comprehensive superiority across all metrics: it reduces GPU memory usage by 27.6%, decreases FLOPs by 23.1%, and achieves a 6.6% performance lead in NDCG@10. Although EulerFormer and LRURec exhibit relatively low computational FLOPs, they still require many GPU memory while delivering subpar performance. Overall, whether compared to linear attention models (LinRec, EulerFormer), Mamba-based architectures (EchoMamba4Rec, SIGMA), or RNN-based approaches (RecBLR, LRURec), RecGRELA achieves a balance between performance and efficiency.

4.4. Ablation Study

4.4.1. Core Components

To investigate how different components contribute to our design, we evaluate RecGRELA variants by removing the key components one by one and implementing some modifications as follows:

- **w/ V Matrix, w/o Input Gate**: Keep the value V and remove the input linear projection before RELA.
- **w/ V Matrix, w/o SiGLU w/**: Remove the SiLU-based gate and keep the value V, this variant verifise the effectiveness of the gated mechanism.

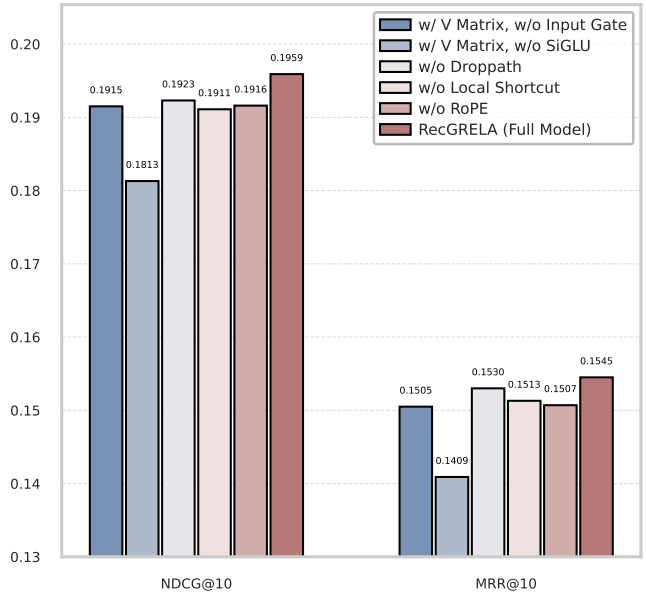

Figure 4: The effectiveness of each variant of RecGRELA on ML-1M.

- **w/o Droppath**: Remove the Droppath operation in the GRELA block.
- **w/o RoPE**: Remove the RoPE in the linear attention.
- **w/o Local Shortcut**: Remove the Local shortcut from the linear attention.

Fig. 4 records two evaluation metrics, i.e., NDCG@10, and MRR@10 on the **ML-1M** dataset. From this figure, we have the following observations:

- The removal of the gating mechanism while retaining the value matrix leads to the most significant performance degradation (e.g., HR@10 drops from 0.3311 to 0.3134 on HR@10), explicitly highlighting the significant role of the SiLU-based gating mechanism in harmonizing long-range sequence dependencies with local preference modeling.
- Removing the local shortcut also induces substantial performance degradation (e.g., HR@10 drops by 1%

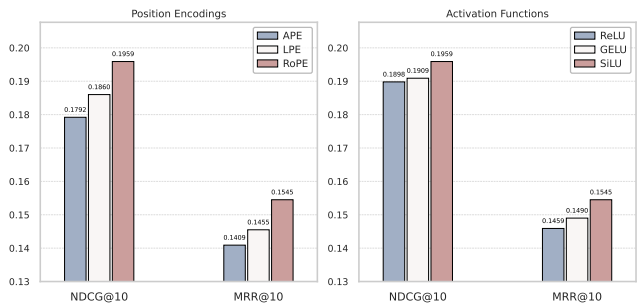

Figure 5: The performance of different position encodings and activation functions.

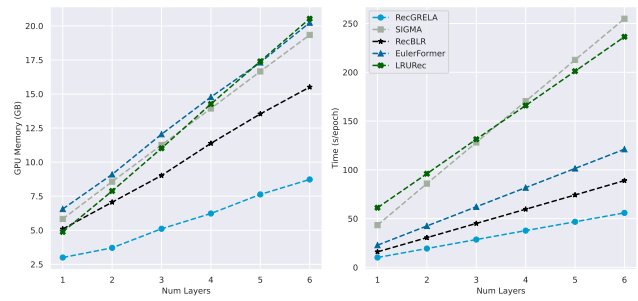

Figure 6: Impact of different stacked layers on computational costs. We set the sequence length to 100 to avoid GPU memory errors.

compared to the full model), demonstrating that incorporating local preferences in long-sequence recommendation systems significantly enhances overall recommendation quality.

- The removal of RoPE incurs moderate performance degradation, demonstrating that RoPE enables efficient modeling of relative positional relationships within user interaction sequences.

- Removing any of the three core contributions in RecGRELA results in substantial performance degradation, particularly in ranking-oriented metrics such as NDCG@10 and MRR@10. This aligns with the observations in Section 4.2.

4.4.2. Position Encoding and Activation Function

We further investigate the impact of position encoding strategies on RecGRELA. As shown in Fig. 5, the proposed RoPE integration demonstrates superior performance across all metrics while achieving faster training speeds and lower memory consumption than alternative position encoding methods. These empirical results validate the effectiveness of RoPE in enhancing representational capacity within the RecGRELA framework.

We also evaluate the impact of different activation functions on the model's performance. We replace the function in the gated mechanism and the MLP. As shown in Fig. 5, the SiLU activation function outperforms ReLU and GELU across all metrics, demonstrating the superior performance of SiLU in capturing complex user behavior and enhancing the model's representational capacity.

4.5. Sensitivity Analysis

In this section, we conduct experiments on the **ML-1M** dataset to analyze the influence of three hyperparameters: the sequence length N, the number of stacked GRELA blocks L, the number of attention heads h, and the Dropout rates.

4.5.1. The Number of Layers

The results in Table 3 reveal that stacking more RecGRELA layers fails to consistently enhance recommendation quality while markedly degrading computational efficiency, primarily due to the overfitting problem of multiple

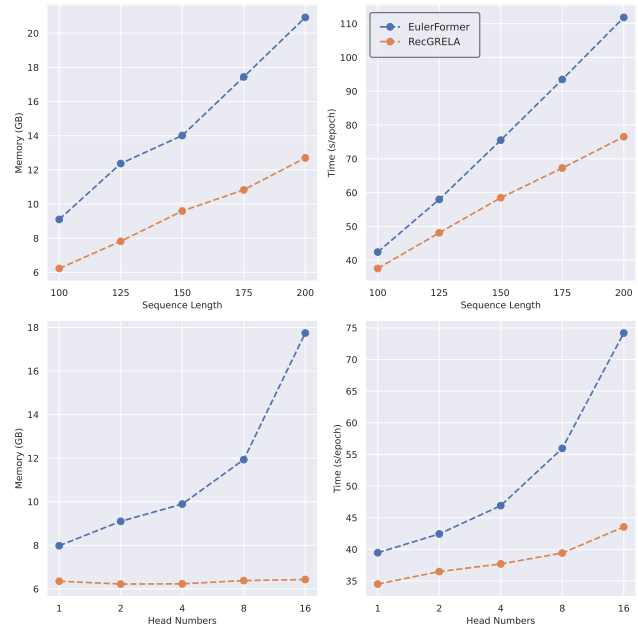

Figure 7: Impact of sequence length and attention head variations on computational costs. When investigating the impact of sequence length, we set the attention head to 2 for EulerFormer and 4 for RecGRELA. When investigating the impact of attention heads, we set the sequence length to 100 to avoid GPU memory errors. Other hyperparameters are the same as the main experiments.

RecGRELA layers. Specifically, model performance peaks at four stacked layers, beyond which further depth expansion leads to metric degradation with substantial computational overhead. Notably, the three-layer configuration achieves competitive performance with state-of-the-art baselines, demonstrating the architecture's effectiveness in balancing representational capacity and operational efficiency. Furthermore, our comparative analysis of layer-wise efficiency between RecGRELA and SIGMA reveals distinct scaling patterns. As illustrated in Fig. 6, the growth trend of computational occupation of RecGRELA is relatively gradual, whereas that of SIGMA shows a steeper increase, indicating that RecGRELA exhibits better scaling capability when the training requires more layers.

4.5.2. *The Number of Sequence Length*

To examine training cost variations across sequence lengths in long-term SRSs, we evaluate maximum sequence lengths from 100 to 200 with 25-step increments. Fig. 7 demonstrates that RecGRELA maintains consistently lower computational complexity than EulerFormer across all tested configurations.

To evaluate the model's performance characteristics with varying input lengths, we varied the maximum sequence length processed by the model, testing values from 100 to 200, while keeping other hyperparameters constant. The impact on recommendation performance was measured using NDCG@10 and MRR@10, as illustrated in Fig. 8. The green and blue lines show the model's performance exhibits relative stability for sequence lengths between 100 and 150. Notably, performance significantly improves when the sequence length increases to 200. This suggests that while the model performs consistently well with shorter sequences, capturing longer-range dependencies present in sequences of length 200 yields a clear benefit in terms of recommendation performance.

4.5.3. *The Number of Attention Heads*

We evaluate the impact of the number of attention heads on model efficiency. As shown in Fig. 7, the results demonstrate that RecGRELA maintains stable GPU memory and training time across varying head counts, while EulerFormer exhibits proportional increases in computational overhead with expanded attention heads. RecGRELA applies rotary position encoding directly to the full embedding dimension, avoiding repetitive computations across attention heads as EulerFormer does. Our method proves that handling position encodings at the embedding level saves computation compared to per-head processing.

We further investigated the model's performance with respect to the number of attention heads. The results are also shown in Fig. 8. Both metrics consistently demonstrate that using four attention heads yields the best performance for our model configuration on this dataset. Utilizing fewer heads (two) appears insufficient to capture the complexity of user-item interactions while employing significantly more heads does not offer further benefits and may even slightly degrade performance. This could be attributed to potential redundancy or fragmentation of attention patterns with excessive heads. Therefore, four attention heads represent the optimal setting identified in our experiments.

4.5.4. *The Dropout and Droppath Rates*

Regularization techniques like Dropout and Droppath prevent overfitting and improve model generalization. We assessed the model's sensitivity to the rates of these two techniques, varying each from 0.1 to 0.5. Fig. 8 displays the impact on NDCG@10 and MRR@10. The sensitivity analysis suggests that lower regularization rates (0.1 for both) are preferable. Higher rates, particularly for Dropout, lead to substantial performance degradation. This highlights the importance of carefully tuning regularization hyperparameters, as overly aggressive regularization can harm performance more than it helps prevent overfitting in sequential recommendation.

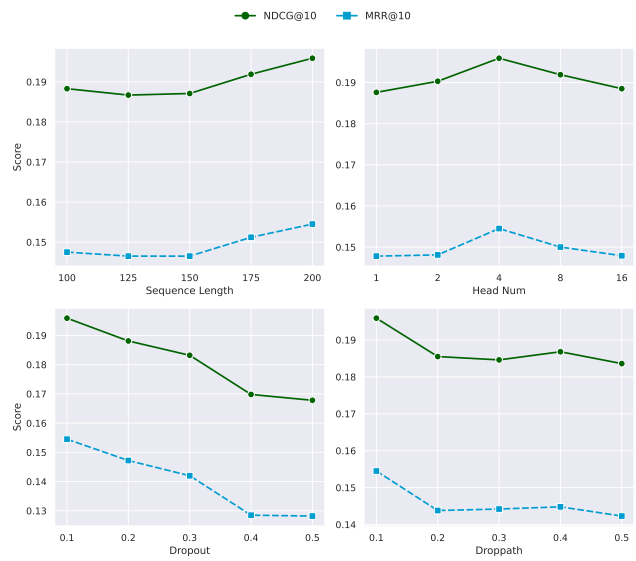

Figure 8: Influence of the sequence length, head numbers, Dropout, and Droppath on various metrics.

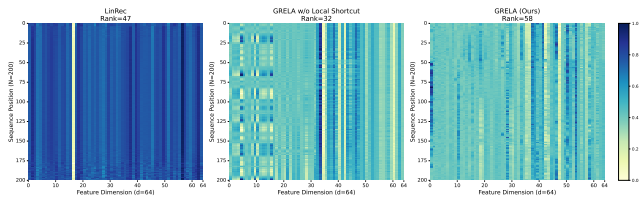

Figure 9: Heatmap of output feature maps from the last layer.

4.6. Visualization Analysis

To further investigate the impact of the proposed Local Shortcut on the representational capacity of our model, particularly regarding the theoretical motivation to increase feature matrix rank, we conduct a visualization analysis on the **ML-1M** dataset. We visualize the output feature maps from the last layer for three distinct models: first, LinRec, which serves as a standard linear attention baseline; second, GRELA without the Local Shortcut, representing our architecture with this specific component ablated; and third, the complete GRELA model, labeled as Ours. The resulting heatmaps display the feature matrix structure for a sequence length N of 200 and a feature dimension d of 64. We also compute the rank for each matrix. Given the feature dimension $d = 64$, this is the maximum possible rank in this configuration.

Fig. 9 helps show why our design is effective by visualizing the resulting feature matrices. The heatmap on the left (LinRec), shows a feature matrix with a rank of 47. The middle heatmap is for GRELA without the Local Shortcut, yielding a feature matrix with the lowest rank at just 32.

Visually, strong horizontal lines and repeating patterns indicate that the features are too similar and lack local detail. This low matrix rank confirms reduced expressive power and information content. In contrast, the heatmap on the right, from our full GRELA model with the Local Shortcut, displays a much more complex and detailed feature matrix. The strong lines disappear, showing more varied features. Significantly, its matrix rank increases substantially to 58, closely approaching the theoretical maximum rank of 64.

The Local Shortcut, which uses causal convolution, adds important local details to the features. This helps fix the low-matrix rank problem that happens with just linear attention. Adding these local details makes the overall matrix rank higher. As a result, the features become richer and show more variety. We can see this improvement in the heatmap, and the higher matrix rank confirms it. This proves the Local Shortcut works well to get around the limits of linear attention. It helps the model capture immediate local preferences and crucial long-term dependencies within the user's interaction sequence.

5. Related Work

5.1. Sequential Recommendation

Earlier fundamental models of sequential recommendation focus on using Convolutional Neural Networks [49, 61] and Recurrent Neural Networks (RNNs) [18, 25]. Due to RNNs' performance limits and poor parallel processing, recent sequential recommendation studies have increasingly turned to Transformer-based architectures [57, 28, 5, 8, 64, 65, 24] or MLPs [33, 67, 27] for better efficiency and scalability. The most representative work is SASRec [19], which is the first to use the self-attention mechanism for capturing user interest features from interaction sequences. BERT4Rec [47] adopts a deep encoder-based bidirectional model with self-attention to model user behavior sequences, achieving strong recommendation performance. However, the Transformer architecture suffers from quadratic computational complexity with respect to sequence length due to the dot-product attention operation, which limits its efficiency for sequence modeling tasks. Recently, many models with linear computational complexity have emerged as alternatives to the original Transformer architecture, such as linear attention [31, 50], linear RNNs [29, 62], and state space models [30, 40, 6]. While LinRec [31] adopts linear attention to reduce the computational complexity of the dot-product attention, they still rely on learnable position encodings to capture positional information. EulerFormer [50] introduces a novel Euler transformation with a rotary form for unified modeling of both semantic and positional difference, yet uses the dot-product attention, which results in computational costs comparable to LinRec. To overcome these specific limitations, RecGRELA combines linear attention with rotary position encodings and introduces a gated mechanism to model both local and long-term user interests.

5.2. Linear Attention

As opposed to dot-product attention, linear attention [20] replaces the Softmax with separate kernel functions, thereby reducing computational complexity to $\mathcal{O}(N)$ through a change in computation order. Though efficient, designing a linear attention module as effective as softmax attention is a nontrivial problem. Although linear attention has achieved some success in other domains, such as computer vision [44, 38, 14, 15] and neural language processing [55, 39, 60]. In sequential recommendation, despite existing linear attention-based methods [7, 31] decreasing the computational complexity, they cannot effectively model position encoding within behavior sequence and fail to capture local preference in long-term interactions. This motivates us to design a more efficient and effective linear attention-based module to address these issues. In response, we present RecGRELA, which aims to realize the full potential of linear attention in sequential recommendation by overcoming these critical challenges in positional awareness and local preference modeling.

6. Conclusion and Future Work

In this work, we propose RecGRELA, a new paradigm for long-term sequential recommendation. By integrating rotary position encoding into a linear attention mechanism, our Rotary-Enhanced Linear Attention module captures long-range dependencies with reduced computational complexity. To model local interaction in long-range dependencies, we design a Local Shortcut operation and provide theoretical insight from the perspective of the rank of the matrix. To address the difficulty of the model in perceiving and adapting to dynamic user preference shifts, we propose a SiLU-based gating mechanism that dynamically balances local and long-term representations. Extensive experiments on five benchmark datasets show that RecGRELA outperforms state-of-the-art methods based on RNNs, Transformers, and Mamba architectures, achieving superior performance with significantly lower memory overhead.

In the future, we plan to explore the application of RecGRELA in other recommendation scenarios, such as session-based and multi-modal recommendations. Since linear attention shows great potential in various domains, we also plan to explore more efficient attention mechanisms for sequential recommendation tasks.

CRediT authorship contribution statement

Wei Zhou: Methodology, Writing – original draft, Validation, Resources. **Huayi Shen:** Methodology, Writing – original draft, Investigation, Software. **Xiao Du:** Supervision, Visualization, Validation, Software. **Jie Liao:** Methodology, Investigation, Software. **Min Gao:** Formal analysis. **Jun Zeng:** Validation, Investigation. **Junhao Wen:** Supervision.

References

[1] Chen, X., Xu, H., Zhang, Y., Tang, J., Cao, Y., Qin, Z., Zha, H., 2018. Sequential recommendation with user memory networks, in: Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining, pp. 108–116.

[2] Clevert, D.A., 2015. Fast and accurate deep network learning by exponential linear units (elus). arXiv preprint arXiv:1511.07289 .

[3] Cui, Q., Wu, S., Liu, Q., Zhong, W., Wang, L., 2018. Mv-rnn: A multi-view recurrent neural network for sequential recommendation. IEEE Transactions on Knowledge and Data Engineering 32, 317–331.

[4] Donkers, T., Loepp, B., Ziegler, J., 2017. Sequential user-based recurrent neural network recommendations, in: Proceedings of the Eleventh ACM Conference on Recommender Systems, p. 152–160.

[5] Du, X., Yuan, H., Zhao, P., Qu, J., Zhuang, F., Liu, G., Liu, Y., Sheng, V.S., 2023. Frequency enhanced hybrid attention network for sequential recommendation, in: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 78–88.

[6] Fan, H., Zhu, M., Hu, Y., Feng, H., He, Z., Liu, H., Liu, Q., 2024. Tim4rec: An efficient sequential recommendation model based on time-aware structured state space duality model. arXiv preprint arXiv:2409.16182 .

[7] Fan, X., Liu, Z., Lian, J., Zhao, W.X., Xie, X., Wen, J.R., 2021. Lighter and better: Low-rank decomposed self-attention networks for next-item recommendation, in: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, p. 1733–1737.

[8] Fan, Z., Liu, Z., Wang, Y., Wang, A., Nazari, Z., Zheng, L., Peng, H., Yu, P.S., 2022. Sequential recommendation via stochastic self-attention, in: Proceedings of the ACM Web Conference, p. 2036–2047.

[9] Fang, H., Zhang, D., Shu, Y., Guo, G., 2020. Deep learning for sequential recommendation: Algorithms, influential factors, and evaluations. ACM Trans. Inf. Syst. 39.

[10] Fu, D.Y., Dao, T., Saab, K.K., Thomas, A.W., Rudra, A., Re, C., 2023. Hungry hungry hippos: Towards language modeling with state space models, in: The Eleventh International Conference on Learning Representations.

[11] Greff, K., Srivastava, R.K., Koutník, J., Steunebrink, B.R., Schmidhuber, J., 2017. Lstm: A search space odyssey. IEEE Transactions on Neural Networks and Learning Systems 28, 2222–2232.

[12] Gu, A., Dao, T., 2024. Mamba: Linear-time sequence modeling with selective state spaces, in: First Conference on Language Modeling.

[13] Han, D., Pan, X., Han, Y., Song, S., Huang, G., 2023. Flatten transformer: Vision transformer using focused linear attention, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 5961–5971.

[14] Han, D., Wang, Z., Xia, Z., Han, Y., Pu, Y., Ge, C., Song, J., Song, S., Zheng, B., Huang, G., 2024a. Demystify mamba in vision: A linear attention perspective, in: Advances in Neural Information Processing Systems, pp. 127181–127203.

[15] Han, D., Ye, T., Han, Y., Xia, Z., Pan, S., Wan, P., Song, S., Huang, G., 2024b. Agent attention: On the integration of softmax and linear attention, in: European Conference on Computer Vision, Springer. pp. 124–140.

[16] He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition.

[17] He, R., Kang, W.C., McAuley, J., 2017. Translation-based recommendation, in: Proceedings of the Eleventh ACM Conference on Recommender Systems, p. 161–169.

[18] Hidasi, B., 2016. Session-based recommendations with recurrent neural networks, in: International Conference on Learning Representations.

[19] Kang, W.C., McAuley, J., 2018. Self-attentive sequential recommendation, in: IEEE International Conference on Data Mining, pp. 197–206.

[20] Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F., 2020. Transformers are rnns: fast autoregressive transformers with linear attention, in: Proceedings of the 37th International Conference on Machine Learning.

[21] Kingma, D.P., 2014. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980 .

[22] Larsson, G., Maire, M., Shakhnarovich, G., 2017. Fractalnet: Ultra-deep neural networks without residuals, in: International Conference on Learning Representations.

[23] Lei Ba, J., Kiros, J.R., Hinton, G.E., 2016. Layer normalization. ArXiv e-prints , arXiv–1607.

[24] Li, C., Wang, Y., Liu, Q., Zhao, X., Wang, W., Wang, Y., Zou, L., Fan, W., Li, Q., 2023a. Strec: Sparse transformer for sequential recommendations, in: Proceedings of the 17th ACM Conference on Recommender Systems, p. 101–111.

[25] Li, J., Ren, P., Chen, Z., Ren, Z., Lian, T., Ma, J., 2017. Neural attentive session-based recommendation, in: Proceedings of the 26th ACM on Conference on Information and Knowledge Management, p. 1419–1428.

[26] Li, J., Wang, Y., McAuley, J., 2020. Time interval aware self-attention for sequential recommendation, in: Proceedings of the 13th International Conference on Web Search and Data Mining, p. 322–330.

[27] Li, M., Zhang, Z., Zhao, X., Wang, W., Zhao, M., Wu, R., Guo, R., 2023b. Automlp: Automated mlp for sequential recommendations, in: Proceedings of the ACM Web Conference, p. 1190–1198.

[28] Li, Y., Chen, T., Zhang, P.F., Yin, H., 2021. Lightweight self-attentive sequential recommendation, in: Proceedings of the 30th ACM International Conference on Information and Knowledge Management, pp. 967–977.

[29] Liu, C., Lin, J., Liu, H., Wang, J., Caverlee, J., 2024a. Behavior-dependent linear recurrent units for efficient sequential recommendation, in: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, pp. 1430–1440.

[30] Liu, C., Lin, J., Wang, J., Liu, H., Caverlee, J., 2024b. Mamba4rec: Towards efficient sequential recommendation with selective state space models. arXiv preprint arXiv:2403.03900 .

[31] Liu, L., Cai, L., Zhang, C., Zhao, X., Gao, J., Wang, W., Lv, Y., Fan, W., Wang, Y., He, M., et al., 2023. Linrec: Linear attention mechanism for long-term sequential recommender systems, in: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 289–299.

[32] Liu, Z., Liu, Q., Wang, Y., Wang, W., Jia, P., Wang, M., Liu, Z., Chang, Y., Zhao, X., 2025. Sigma: Selective gated mamba for sequential recommendation, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 12264–12272.

[33] Long, C., Yuan, H., Fang, J., Xian, X., Liu, G., Sheng, V.S., Zhao, P., 2024. Learning global and multi-granularity local representation with mlp for sequential recommendation. ACM Trans. Knowl. Discov. Data 18.

[34] Lopez-Avila, A., Du, J., Shimary, A., Li, Z., 2024. Positional encoding is not the same as context: A study on positional encoding for sequential recommendation. arXiv preprint arXiv:2405.10436 .

[35] Niu, J., Zhou, W., Luo, F., Zhang, Y., Zeng, J., Wen, J., 2025. Intent-guided bilateral long and short-term information mining with contrastive learning for sequential recommendation. IEEE Transactions on Services Computing 18, 212–225.

[36] Orvieto, A., Smith, S.L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., De, S., 2023. Resurrecting recurrent neural networks for long sequences, in: Proceedings of the 40th International Conference on Machine Learning.

[37] Pan, L., Pan, W., Wei, M., Yin, H., Ming, Z., 2024. A survey on sequential recommendation. arXiv preprint arXiv:2412.12770 .

[38] Qin, Z., Sun, W., Deng, H., Li, D., Wei, Y., Lv, B., Yan, J., Kong, L., Zhong, Y., 2022. cosformer: Rethinking softmax in attention, in: International Conference on Learning Representations.

[39] Qin, Z., Yang, S., Zhong, Y., 2023. Hierarchically gated recurrent neural network for sequence modeling, in: Proceedings of the 37th International Conference on Neural Information Processing Systems.

[40] Qu, H., Zhang, Y., Ning, L., Fan, W., Li, Q., 2024. Ssd4rec: a structured state space duality model for efficient sequential recommendation. arXiv preprint arXiv:2409.01192 .

[41] Ramachandran, P., Zoph, B., Le, Q.V., 2017. Searching for activation functions. arXiv preprint arXiv:1710.05941 .

[42] Rendle, S., Freudenthaler, C., Schmidt-Thieme, L., 2010. Factorizing personalized markov chains for next-basket recommendation, in: Proceedings of the 19th International Conference on World Wide Web, p. 811–820.

[43] Shaw, P., Uszkoreit, J., Vaswani, A., 2018. Self-attention with relative position representations, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers), pp. 464–468.

[44] Shen, Z., Zhang, M., Zhao, H., Yi, S., Li, H., 2021. Efficient attention: Attention with linear complexities, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, pp. 3531–3539.

[45] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R., 2014. Dropout: a simple way to prevent neural networks from overfitting. J. Mach. Learn. Res. 15, 1929–1958.

[46] Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., Liu, Y., 2024. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing 568, 127063.

[47] Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., Jiang, P., 2019. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer, in: Proceedings of the 28th ACM International Conference on Information and Knowledge Management, p. 1441–1450.

[48] Sun, X., Sun, F., Zhang, Z., Li, P., Wang, S., 2024. Adaptive self-supervised learning for sequential recommendation. Neural Networks 179, 106570.

[49] Tang, J., Wang, K., 2018. Personalized top-n sequential recommendation via convolutional sequence embedding, in: Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining, p. 565–573.

[50] Tian, Z., Zhao, W.X., Zhang, C., Zhao, X., Ma, Z., Wen, J.R., 2024. Eulerformer: Sequential user behavior modeling with complex vector attention, in: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, p. 1619–1628.

[51] Van Der Westhuizen, J., Lasenby, J., 2018. The unreasonable effectiveness of the forget gate. arXiv preprint arXiv:1804.04849 .

[52] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, p. 6000–6010.

[53] Wang, B., Shang, L., Lioma, C., Jiang, X., Yang, H., Liu, Q., Simonsen, J.G., 2021. On position embeddings in bert, in: International Conference on Learning Representations.

[54] Wang, S., Hu, L., Wang, Y., Cao, L., Sheng, Q.Z., Orgun, M., 2019. Sequential recommender systems: Challenges, progress and prospects, in: Proceedings of the 28th International Joint Conference on Artificial Intelligence, pp. 6332–6338.

[55] Wang, S., Li, B.Z., Khabsa, M., Fang, H., Ma, H., 2020. Linformer: Self-attention with linear complexity. arXiv preprint arXiv:2006.04768 .

[56] Wang, Y., He, X., Zhu, S., 2024. Echomamba4rec: Harmonizing bidirectional state space models with spectral filtering for advanced sequential recommendation. arXiv preprint arXiv:2406.02638 .

[57] Wu, L., Li, S., Hsieh, C.J., Sharpnack, J., 2020. Sse-pt: Sequential recommendation via personalized transformer, in: Proceedings of the 14th ACM Conference on Recommender Systems, p. 328–337.

[58] Xu, C., Zhao, P., Liu, Y., Xu, J., S.Sheng, V.S., Cui, Z., Zhou, X., Xiong, H., 2019. Recurrent convolutional neural network for sequential recommendation, in: Proceedings of the ACM Web Conference, p. 3398–3404.

[59] Xu, L., Tian, Z., Zhang, G., Zhang, J., Wang, L., Zheng, B., Li, Y., Tang, J., Zhang, Z., Hou, Y., Pan, X., Zhao, W.X., Chen, X., Wen, J.R., 2023. Towards a more user-friendly and easy-to-use benchmark library for recommender systems, in: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, p. 2837–2847.

[60] Yang, S., Wang, B., Shen, Y., Panda, R., Kim, Y., 2024. Gated linear attention transformers with hardware-efficient training, in: Proceedings of the 41st International Conference on Machine Learning.

[61] Yuan, F., He, X., Karatzoglou, A., Zhang, L., 2020. Parameter-efficient transfer from sequential behaviors for user modeling and recommendation, p. 1469–1478.

[62] Yue, Z., Wang, Y., He, Z., Zeng, H., Mcauley, J., Wang, D., 2024. Linear recurrent units for sequential recommendation, in: Proceedings of the 17th ACM International Conference on Web Search and Data Mining, p. 930–938.

[63] Zhang, T., Zhao, P., Liu, Y., Sheng, V.S., Xu, J., Wang, D., Liu, G., Zhou, X., 2019. Feature-level deeper self-attention network for sequential recommendation, in: Proceedings of the 28th International Joint Conference on Artificial Intelligence, p. 4320–4326.

[64] Zhang, Y., Wang, X., Chen, H., Zhu, W., 2023. Adaptive disentangled transformer for sequential recommendation, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, p. 3434–3445.

[65] Zhao, K., Zou, L., Zhao, X., Wang, M., Yin, D., 2023. User retention-oriented recommendation with decision transformer, in: Proceedings of the ACM Web Conference, p. 1141–1149.

[66] Zhao, W.X., Mu, S., Hou, Y., Lin, Z., Chen, Y., Pan, X., Li, K., Lu, Y., Wang, H., Tian, C., Min, Y., Feng, Z., Fan, X., Chen, X., Wang, P., Ji, W., Li, Y., Wang, X., Wen, J.R., 2021. Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms, in: Proceedings of the 30th ACM International Conference on Information and Knowledge Management, p. 4653–4664.

[67] Zhou, K., Yu, H., Zhao, W.X., Wen, J.R., 2022. Filter-enhanced mlp is all you need for sequential recommendation, in: Proceedings of the ACM Web Conference, p. 2388–2399.

[68] Zhu, X., Li, L., Liu, W., Luo, X., 2024. Multi-level sequence denoising with cross-signal contrastive learning for sequential recommendation. Neural Networks 179, 106480.